

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026 umhackathon@um.edu.my

Introduction:

This section introduces the strategic solution of the identified problem statement highlighted in the Product Review Documentation. “UM CRM” aims to modernize manual and fragmented sales tracking by providing an intelligent, AI-powered CRM platform that automates data entry and task management directly through natural language interfaces like WhatsApp and Telegram.

Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the UM CRM development, specifically focusing on the multi-actor AI agent integration.

So, the key elements of the system that have been covered are as followed:

Architecture:

It’s a cloud-native platform. The services are separated into a React-based frontend and a FastAPI microservice backend. Deployed on Fly.io, these services work independently but communicate via REST APIs, leveraging a unified PostgreSQL database for persistent storage.

Data Flows:

How the data flows from a Customer’s mobile device via WhatsApp, moves through the FastAPI Webhook, gets parsed by the LangGraph AI Agent (Gatekeeper -> Manager -> Worker), gets stored in PostgreSQL in a persistent manner, and finally returns a natural language confirmation message back to the user.

Model Process:

The model shows the end-to-end workflow of the core features: receiving a natural language prompt, interpreting intent, assigning specific CRM tool tasks (like creating a contact or logging a meeting), executing database updates, and confirming the action.

Role of Reference:

This document is a reference for all the developers and testers to enable the overall high-level architecture of this project, particularly focusing on the AI agent’s behavior and system dependencies.

Background:

Many growing sales teams and SMEs struggle with manually updating their pipelines using disparate spreadsheets and siloed communication tools. They found a market where tracking interactions, updating deal stages, and managing contacts is overly manual, leading to data loss and decreased sales velocity.

Previous Version:

The existing operations are heavily reliant on manual data entry across Microsoft Excel and scattered chat applications (WhatsApp/Telegram). It requires sales representatives to switch contexts continuously between chatting with clients and updating their local spreadsheets, lacking a unified “Single Source of Truth.”

Changes in Major Architectural Components

New capabilities introduced in this system include a central unified database (PostgreSQL), an interactive web-based Kanban board (Next.js), and most importantly, an autonomous LangGraph AI service layer capable of translating unstructured chat messages into structured CRM operations.

Target Stakeholder

Stakeholders	Roles	Expectations
Sales Representative	Manages contacts, updates opportunity pipelines, and reviews logged activities via the Web Dashboard.	Intuitive Kanban board interface, automated activity logging, and real-time synchronization between chat and dashboard.
Customer/Client	Interacts with the business via WhatsApp/Telegram to inquire about deals, schedule meetings, or ask questions.	Quick, natural language responses without needing to navigate a separate portal or app.

Stakeholders	Roles	Expectations
Account Manager	Oversees the overall sales pipeline and assigns opportunities to different representatives.	Accurate reporting, zero orphaned deals, and immediate notification of high-value opportunity updates.
Development Team	Builds the platform, configures the LangGraph agent, and maintains the Fly.io deployment.	Clear API usage contracts, modular codebase for the AI agent (Gatekeeper/Manager/Worker nodes), and well-documented architecture.
QA Team	Validates system behavior, load tests webhooks, and tests prompt boundaries.	Defined scope, clear AI intent mapping boundaries, and coverage of edge cases like prompt injection and hallucination.

System Architecture & Design

High Level Architecture

Overview:

Type	Details
System	Web Dashboard (React/Next.js) & Chat Integration (WhatsApp/Telegram)
Architecture	Cloud-Server with AI Microservice Layer

UM CRM is primarily structured as a client-server-based system interacting with external chat clients. 1. **Web Client:** Next.js Dashboard used by Sales Reps. 2. **Chat Client:** Customers/Users on WhatsApp/Telegram. 3. **Backend API:** FastAPI serving REST endpoints and Webhooks.

These clients communicate via the Backend API layer. The backend houses the LangGraph AI Agent. All services are deployed on Fly.io (Singapore region) and share a PostgreSQL database.

LLM as Service Layer

The architecture of UM CRM integrates OpenRouter/Ilmu AI as a core service layer, managed through a LangGraph orchestration framework.

Dependency Diagram - Prompt Construction: Prompts are constructed dynamically within the LangGraph state. The state dictionary (**AgentState**) accumulates **user_input**, **contact_name**, **business_context**, and a **messages** history list. - **Context Window:** The context includes the user's immediate message, relevant historical chat messages, and strict CRM tool schemas (Pydantic models). - **Parsing & Flow:** 1. The webhook receives a JSON payload. 2. The message is passed to the **Gatekeeper Node** (Intent router). If it's a simple greeting (e.g., "Hi"), a fast-path bypasses the LLM to save tokens and latency. Otherwise, it queries the LLM for intent. 3. If CRM action is needed, the **Manager Node** (Workflow Planner) decomposes the prompt into structured JSON tasks. 4. The **Worker Node** (Execution Engine) parses the JSON tasks and triggers local Python CRM tools (e.g., **create_contact**). - **Token Limitations:** Token limits are enforced prior to the LLM call. Webhook payloads exceeding 2000 tokens are chunked or rejected with a 413 Payload Too Large error to prevent exceeding OpenRouter/Ilmu AI context limits. - **API Interactions:** Customer WhatsApp -> WhatsApp API -> FastAPI Webhook (/api/whatsapp) -> LangGraph Agent -> OpenRouter/Ilmu AI API -> PostgreSQL -> FastAPI -> WhatsApp API.

Sequence Diagram Place Order / Log Activity Flow: 1. **Customer** sends: "I'm John from Acme. Let's schedule a meeting tomorrow." via WhatsApp. 2. **WhatsApp API** pushes the payload to the FastAPI /api/whatsapp webhook. 3. **FastAPI** initializes the LangGraph **AgentState**. 4. **Gatekeeper Node** analyzes the prompt, identifying it's not a simple greeting and requires CRM action. Sets **agent_loop = True**. 5. **Manager Node** queries the LLM and outputs a JSON list: [{"tool": "create_contact", "args": {"name": "John", "company": "Acme"}}, {"tool": "create_activity", "args": {"type": "meeting"}}]. 6. **Worker Node** executes these tools, executing INSERT statements into the PostgreSQL database. 7. **Worker Node** updates the **AgentState** with the success results. 8. **Manager Node** generates a natural language summary: "I've added you to our contacts and logged our meeting for tomorrow!" 9. **FastAPI** returns the response to the WhatsApp API, which delivers it to the customer.

Technological Stack

- **Frontend:** Next.js 14 (App Router) – React, Tailwind CSS, @dnd-kit for Kanban drag-and-drop.
- **Backend:** FastAPI (Python) – High-performance async RESTful API and webhook handler.
- **AI Orchestration:** LangGraph & LangChain Core – Manages stateful, multi-actor AI workflows.

- **LLM Provider:** OpenRouter / Ilmu AI – Core reasoning engines.
- **Database:** PostgreSQL – Relational Database for users, contacts, opportunities, and activities (using SQLAlchemy ORM).
- **Cloud/Deployment:** Fly.io (Dockerized Containers) with AWS RDS (or Fly Postgres) for database persistence.

Key Data Flows

Data Flow Diagram (DFD) - External Entity (User) -> Sends Message -> Process 1 (Webhook Listener). - **Process 1 -> Updates State -> Process 2 (LangGraph Gatekeeper/Manager).** - **Process 2 -> Requests LLM Inference -> External Service (Ilmu AI).** - **Process 2 -> Generates Tool Tasks -> Process 3 (Worker Execution).** - **Process 3 -> Reads/Writes Data -> Database (PostgreSQL).** - **Process 3 -> Returns Status -> Process 1 -> Sends Reply -> External Entity (User).**

Normalized Database Schema (3NF) - users: Stores system users (Sales Reps) with hashed passwords and JWT details. - **contacts:** Stores customer details (name, email, phone, company). Resolves 1:N relationship with users. - **opportunities:** Stores deal tracking (title, value, stage). Junctions to contacts and users. - **activities:** Logs interactions (type, notes, timestamp). Junctions to contacts.

Functional Requirements & Scope

Minimum Viable Product:

#	Features	Description
1	AI Chatbot Integration	Customers can interact with the CRM via WhatsApp/Telegram. The AI parses intents and answers queries using the LangGraph engine.
2	Automated CRM Task Execution	The LangGraph Worker node autonomously creates contacts, logs activities, and updates deal stages based on chat context.
3	Kanban Opportunity Board	A Next.js web dashboard allowing Sales Reps to visually drag-and-drop opportunities across different sales stages.
4	Contact Management	A centralized directory to view customer details, historical activities, and associated opportunities.

Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	The system must handle bursts of webhook traffic during peak sales hours without dropping messages.	Use Fly.io’s auto-start/stop machines and horizontal scaling for the FastAPI backend containers.
Reliability	AI-driven database updates must not result in corrupted or orphaned CRM records if an LLM call fails mid-execution.	Implementation of strict SQLAlchemy transactional rollbacks. If the Worker node fails, the DB transaction is rolled back.
Maintainability	The AI agent logic must be easily modifiable without breaking the web API endpoints.	LangGraph nodes (Gatekeeper, Manager, Worker) are separated into distinct Python modules (<code>cs_agent.py</code> , <code>tools.py</code>).
Token Latency	Webhook responses must return within 1500ms under normal load (p95).	Async API calls. A “fast-path” regex intercepts simple greetings (e.g., “Hi”) and returns immediately without invoking the LLM.
Cost Efficiency	Average LLM token costs per user session must remain low to ensure profitability.	Truncation of chat history to the last 5 messages. Using cheaper, specialized models for the Manager node and avoiding LLM calls for static queries.

Out of Scope / Future Enhancements

- In-App chat features directly within the Next.js dashboard.
- Promotional offers, discount code generation, and automated mass-marketing campaigns.
- Deep, bi-directional syncing with Google Workspace (Gmail/Calendar) beyond the current mock implementations.
- Native iOS/Android applications for the Sales Representatives.

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation:

Grayscale Rollout & A/B Testing: New LangGraph tool capabilities (e.g., adding an “invoice generation” tool) are released to 5% of real webhook traffic initially. State execution logs are monitored for hallucination rates. If no hallucinations or DB errors are detected, traffic is gradually increased to 100%. A/B

testing is conducted on the Manager node’s system prompt to determine which variant yields higher task-parsing accuracy.

Emergency Rollback (ER) & Golden Release: If anomalous behavior is detected (e.g., the Worker node begins failing > 5% of database inserts due to schema mismatch), an Emergency Rollback is triggered via Fly.io. The automated CI/CD pipeline instantly reverts the Docker container to the previous “Golden Version” to restore CRM stability.

Priority Matrix:

- **P0 Critical:** Complete failure of the FastAPI server or PostgreSQL database. Action: Immediate rollback and DB failover.
- **P1 High:** Webhooks timing out or Gatekeeper node consistently rejecting valid intents. Action: Scale up containers or revert prompt changes within 1 hour.
- **P2 Medium:** UI bugs in the Next.js Kanban board (e.g., drag-and-drop visual glitch). Action: Address in the next sprint.

Monitoring:

System health is monitored via Fly.io metrics and internal Python logging modules tracking LangGraph state transitions. Alerts are triggered if the webhook error rate exceeds 2% or if API latency spikes above 2000ms consistently for 5 minutes.

Assumptions:

- Customers have stable internet connections on their devices when sending WhatsApp/Telegram messages.
- The OpenRouter / Ilmu AI endpoints maintain at least 99.9% uptime.
- Sales Representatives have modern web browsers (Chrome/Safari) to access the React-based dashboard.

External Dependencies:

Tools	Purpose	Risks
Ilmu AI / OpenRouter API	Core LLM reasoning engine. Handles prompt inference, multi-step task decomposition, and generating structured JSON tool outputs.	High - Rate limits or API outages will cause the core AI feature to fail. Mitigation: Graceful degradation state, falling back to a standard “Agent Offline” auto-reply.

Tools	Purpose	Risks
WhatsApp/Telegram APIs	External gateways receiving customer messages and forwarding them to our webhooks.	Medium - Changes in webhook payload structures by Meta/Telegram could break our parsing logic.
Fly.io	Cloud hosting for Dockerized frontend and backend services.	Low - Highly reliable, but regional outages (Singapore) could cause temporary downtime.

Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

Project Timeline: - **Day 1-2:** Project Planning, UI/UX Design (Tailwind), and Database Schema Modeling. - **Day 3-5:** FastAPI Backend Setup, PostgreSQL Integration, and CRUD Endpoints. - **Day 6-7:** LangGraph AI Agent Development (Gatekeeper, Manager, Worker nodes) and LLM Prompt Engineering. - **Day 8:** Next.js Frontend Development and Kanban Board implementation. - **Day 9:** Webhook Integration (WhatsApp/Telegram) and End-to-End Testing. - **Day 10:** Deployment to Fly.io, Documentation (SAD, QATD), and Final Review.

Recommendations: For future scaling, implementing a Redis Cache is recommended to store active LangGraph conversational states and session histories, reducing the load on the PostgreSQL database during high-traffic conversational bursts.